

Dokumentasi — Reverse Workflow Platform

Dokumen ini menjelaskan instalasi, penggunaan, alur kerja, dan hal-hal penting yang **wajib diketahui** untuk menjalankan dan memahami sistem hasil refactor (versi tanpa Qdrant, dengan pemetaan reverse eksplisit di Neo4j).

1. Ringkasan Sistem

Reverse Workflow Platform menerima sebuah **workflow Shuffle SOAR** (rangkaian action yang sudah dieksekusi), lalu menghasilkan **reverse workflow** — yaitu workflow pembalik yang menjalankan aksi kebalikan dalam urutan terbalik (mis. `Block IP` → `Unblock IP`), untuk membantu analis melakukan rollback saat terjadi false positive.

Poin penting tentang sifat sistem:

- Keluaran adalah **draft/rekomendasi**, bukan tindakan yang dieksekusi otomatis. Analis tetap meninjau sebelum digunakan.
 - Aksi yang **tidak punya pasangan pembalik** atau bersifat **destruktif/irreversibel** (mis. `Delete File`) tidak dikarang, melainkan ditandai `requires_manual_review`.
 - Seluruh komponen berjalan **lokal** (Docker): Neo4j, PostgreSQL, Ollama. Tidak ada layanan cloud, tidak ada Qdrant/RAG.
-

2. Arsitektur & Komponen

Sistem terdiri dari dua service utama + tiga pendukung, semuanya container Docker dalam satu jaringan.

Komponen	Peran	Image/Bahasa	Port
<code>reverse-workflow-service</code> (rwp-orchestrator)	Orkestrator: parsing, build graph, simpan Neo4j, panggil LLM, validasi, import Shuffle	Node.js	5005
<code>llm-service</code> (rwp-llm)	Query Neo4j, bangun prompt berbasis pemetaan, panggil Ollama	Python/FastAPI	8000
<code>neo4j</code>	Knowledge graph (Workflow/Action/App/ReverseAction)	neo4j:5	7474 (UI), 7687 (bolt)
<code>postgres</code>	Penyimpanan relasional (katalog app, log)	postgres:16	5432
<code>ollama</code>	Runtime model LLM lokal (llama3.1:8b)	ollama/ollama	11434

Jaringan Docker: service memakai key `platform` yang **eksternal** dengan nama nyata `shared-platform` (lihat bagian Instalasi — jaringan ini harus dibuat lebih dulu).

3. Prasyarat

- Docker Engine + Docker Compose v2 (`docker compose`, bukan `docker-compose`).
- RAM memadai untuk Ollama menjalankan `llama3.1:8b`. Pada CPU, satu generasi bisa makan 5-15 menit (timeout default sudah disetel 900 detik). GPU sangat disarankan.
- Ruang disk untuk image + model Ollama (~beberapa GB).
- (Opsional, untuk end-to-end nyata) Instance **Shuffle SOAR** yang bisa diakses, beserta API key — diperlukan agar katalog app tersinkron dan reverse workflow bisa diimpor kembali.

4. Instalasi Langkah demi Langkah

4.1 Ekstrak

```
bash
```

```
unzip reverse-workflow-platform-refactored.zip
cd reverse-workflow-platform
```

4.2 Buat jaringan eksternal (WAJIB, sekali saja)

Compose memakai jaringan eksternal bernama `shared-platform`. Jika belum ada, buat dulu:

```
bash
docker network create shared-platform
```

Jika tidak dibuat, `docker compose up` akan gagal dengan error "network shared-platform declared as external, but could not be found".

4.3 Siapkan environment

```
bash
cp .env.example .env
```

Lalu edit `.env` dan isi minimal:

- `POSTGRES_PASSWORD` — bebas (mis. `admin`).
- `NEO4J_PASSWORD` — bebas, tapi **harus sama** dengan password container Neo4j (lihat 4.5).
- `LLM_SECRET_KEY` — string acak panjang (untuk menandatangani JWT).
- `LLM_AUTH_USER` / `LLM_AUTH_PASS` — default `admin` / `admin` (dipakai orchestrator untuk login ke llm-service).
- `OLLAMA_HOST` — lihat catatan penting di bawah.
- `SHUFFLE_API_URL` / `SHUFFLE_API_KEY` — isi jika ingin sinkron app + import nyata.

Catatan Ollama (PENTING): `.env.example` default `OLLAMA_HOST=http://host.docker.internal:11434` (artinya memakai Ollama yang terinstal di host). Jika Anda memakai **container ollama bawaan** compose, ganti menjadi `OLLAMA_HOST=http://ollama:11434`. Salah satu saja, harus konsisten.

4.4 Jalankan stack

```
bash
```

```
docker compose up -d --build
# atau: make up
```

4.5 Set password Neo4j (sekali, jika diminta)

Container Neo4j memakai kredensial dari environment; pastikan `NEO4J_PASSWORD` di `.env` cocok dengan yang dipakai container. Jika Neo4j meminta ganti password awal, samakan dengan `.env`. URI default: `bolt://neo4j:7687`, user `neo4j`, database `neo4j`.

4.6 Tarik model LLM ke Ollama

```
bash

docker compose exec ollama ollama pull llama3.1:8b
# atau: make pull-model
```

4.7 Cek kesehatan

```
bash

make health
# LLM service → curl http://localhost:8000/health → {"status":"ok",...}
# Orchestrator → curl http://localhost:5005/ → "Reverse Workflow Servi
```

Perintah Makefile berguna lainnya: `make logs` (tail semua log), `make ps`, `make down`, `make clean` (hapus volume — **menghancurkan data**).

5. Konfigurasi Penting (.env)

Variabel	Fungsi	Catatan
<code>NEO4J_URI</code> / <code>NEO4J_USERNAME</code> / <code>NEO4J_PASSWORD</code> / <code>NEO4J_DATABASE</code>	Koneksi Neo4j lokal	host <code>neo4j</code> , user wajib <code>neo4j</code> untuk container bawaan
<code>POSTGRES_*</code>	Koneksi PostgreSQL	dipakai bersama kedua service

Variabel	Fungsi	Catatan
<code>LLM_SECRET_KEY</code>	Penandatanganan JWT di llm-service	jaga stabil antar restart
<code>LLM_AUTH_USER</code> / <code>LLM_AUTH_PASS</code>	Akun login orchestrator → llm-service	default <code>admin</code> / <code>admin</code>
<code>LLM_API_URL</code> / <code>LLM_API_PREFIX</code>	Alamat llm-service dari Node	<code>http://llm-service:8000</code> + <code>/api/v1</code>
<code>LLM_MODEL</code>	Model Ollama	<code>llama3.1:8b</code>
<code>OLLAMA_HOST</code>	Alamat Ollama	container: <code>http://ollama:11434</code> ; host: <code>http://host.docker.internal:11434</code>
<code>OLLAMA_READ_TIMEOUT</code>	Timeout generasi (detik)	default 900 (CPU). Turunkan jika GPU
<code>LLM_CALL_TIMEOUT_MS</code>	Timeout axios Node (ms)	$\approx \text{OLLAMA_READ_TIMEOUT} * 1000 + 60000$
<code>SHUFFLE_API_URL</code> / <code>SHUFFLE_API_KEY</code>	Integrasi Shuffle	untuk sync katalog app + import workflow
<code>APP_SYNC_TTL_HOURS</code>	Masa segar cache katalog app	default 24

6. Cara Penggunaan (Endpoint)

Semua endpoint inti berada di orchestrator (port 5005). Endpoint llm-service (port 8000) bersifat internal (dipanggil orchestrator), kecuali `/health`.

6.1 Hasilkan reverse workflow — `POST /api/reverse-workflow`

Dipanggil oleh Shuffle (atau manual untuk uji). Body berisi workflow sumber:

```
bash
```

```
curl -X POST http://localhost:5005/api/reverse-workflow \
-H "Content-Type: application/json" \
-d '{
  "workflow_id": "1ce4b8f0-0000-0000-0000-000000000001",
  "workflow_name": "Incident Response - Block",
  "actions": [ /* daftar action Shuffle */ ],
  "branches": [ /* daftar branch Shuffle */ ]
}'
```

Respons sukses (ringkas):

```
json
{
  "success": true,
  "workflow_id": "...",
  "generated_workflow_id": "...",
  "generated_workflow_name": "...",
  "attempts": 1,
  "review_required": [ { "id": "...", "name": "manual_review_required" } ],
  "message": "Reverse workflow generated successfully (draft – perlu peninjauan"
}
```

`review_required` memuat action yang ditandai untuk peninjauan analis (mis. hasil dari `Delete File`)).

6.2 Cek status proses — `GET /api/reverse-workflow/status/:id`

```
bash
curl http://localhost:5005/api/reverse-workflow/status/<workflow_id>
# → { "success": true, "workflow_id": "...", "status": "processing|success|failed"
```

6.3 Validasi tanpa import — `POST /api/validate-workflow`

Menjalankan validator terhadap JSON yang dikirim, tanpa mengimpor ke Shuffle:

```
bash
```

```
curl -X POST http://localhost:5005/api/validate-workflow \
  -H "Content-Type: application/json" \
  -d '{ "workflow": { /* reverse workflow JSON */ }, "workflow_id": "..." }'
# → { "success": true, "valid": true|false, "errors": [...], "review_required":
```

7. Alur Kerja End-to-End

```
[Shuffle]
  | POST /api/reverse-workflow {workflow_id, workflow_name, actions,
branches}
  ▼
[server.js (reverse-workflow-service, :5005)]
  1. parseWorkflow()      → ekstrak action, app, parameter, dependency
  2. buildGraph()          → node WORKFLOW/ACTION/APP + REVERSE_ACTION,
                           relasi CONTAINS/USES_APP/NEXT + HAS_REVERSE
                           (status: auto_mapped | needs_llm |
requires_manual_review | no_reverse_needed)
  3. saveGraphToNeo4j()    → simpan ke Neo4j (bolt :7687)
  4. syncAppCatalog() (safe) → sinkron katalog app (Shuffle→Postgres/Neo4j)
  5. getWorkflowContext()  → verifikasi graph + baca pemetaan reverse
  6. generateWithRetry()   ─────────── (login JWT → /api/v1/auth/login)
                           ▼
[generate.py (llm-service, :8000) POST /api/v1/generate/reverse]
  a. _get_workflow_graph() → query Neo4j (Action nodes)
  b. graph_retrieval.get_action_context() → baca HAS_REVERSE
(reverse_action_name+status)
  c. _build_reverse_system_prompt() → prompt berbasis PEMETAAN (bukan RAG):
                                   auto_mapped→pakai reverse_action_name;
                                   needs_llm→LLM simpulkan;
manual→placeholder;
                                   no_reverse_needed→dilewati (tidak
masuk output)
  d. llm.complete()        → Ollama (llama3.1:8b) → raw JSON
  e. inject_large_image + sanitize → kembalikan raw_output ke Node
  ───────────┘
  7. parseWorkflowJSON() → JSON.parse
  8. validateWorkflow(workflow, workflow_id):
      Level A struktural → Level B semantik(APP) → Level C rule reverse-
mapping
```

```
+ kumpulkan review_required
(jika gagal → retry_context dikirim balik ke llm-service, maks 3x)
9. importWorkflowToShuffle() → import draft ke Shuffle
▼
[Respons ke pemanggil: generated_workflow_id + review_required]
▼
[Analisis meninjau draft sebelum digunakan]
```

Inti yang wajib dipahami: **prompt dibangun di Python (llm-service)**, bukan di Node. Node hanya mengirim `workflow_id`, `workflow_name`, dan `retry_context`. Pemetaan reverse dibaca dari relasi `HAS_REVERSE` di Neo4j.

8. Pemetaan Reverse Action (kamus v2, per-aplikasi)

Sumber kebenaran tunggal: `services/reverse-workflow-service/config/reverseActionMap.json` (format **v2**). Kunci pemetaan adalah (**app_name** + **action_name**) memakai nama action asli tiap aplikasi (mis. FortiGate memakai `post_add_firewall_address`, bukan `block_ip`). Struktur:

```
json
{
  "version": 2,
  "heuristics": {
    "no_reverse_prefixes": ["get_", "list_", "search_", "head_", "describe_", "
    "manual_review_prefixes": ["put_set_", "put_update", "patch_", "delete_", "
    "reversible_prefix_pairs": [["post_add_", "delete_"], ["enable_", "disable_
  ],
  "apps": {
    "FortiGate_Firewall": {
      "reversible": [
        { "action": "post_add_firewall_address", "reverse_action": "delete_fire
      ],
      "requires_manual_review": ["post_add_vdom_link_interface"],
      "no_reverse_needed": ["post_login", "post_logout"]
    },
    "Shuffle Tools": { "default_status": "no_reverse_needed" }
  }
}
```

Setiap action diklasifikasikan ke **empat status** saat `buildGraph()` lewat `config/reverseMap.js` (`resolveReverse(actionName, appName)`):

Status	Arti	Di reverse workflow
<code>auto_mapped</code>	ada di <code>reversible</code> app	keluarkan reverse action (<code>name = reverse_action_name</code>)
<code>requires_manual_review</code>	irreversibel / update stateful / tanpa pasangan	placeholder <code>manual_review_required</code>
<code>no_reverse_needed</code>	read-only / utilitas	dilewati (tidak masuk output)
<code>needs_llm</code>	tak terklasifikasi	LLM menyimpulkan; ragu → manual

Urutan resolusi: `apps[app].reversible` → `requires_manual_review` → `no_reverse_needed` → `needs_llm` (daftar per app) → `default_status` app → heuristik prefix global → `needs_llm`. Aplikasi utilitas (Shuffle Tools, Shuffle AI, ShuffleHealthcheck, http, email) memakai `default_status: no_reverse_needed` sehingga seluruh action-nya dilewati tanpa perlu didaftar.

Pencocokan memakai **normalisasi** (lowercase, spasi/`-` → `_`) untuk action maupun app. **Untuk menambah/mengubah pemetaan, cukup edit JSON ini** — tidak perlu mengubah kode. App yang didukung saat ini: FortiGate, AWS Network Firewall, Cisco ASA, Palo Alto, FortiEDR, Sophos Central, Microsoft Entra ID, Wazuh, Datadog, Elasticsearch, FortiSIEM, plus app bawaan Shuffle.

Catatan: `reverse_action` yang dipetakan harus benar-benar ada di katalog app yang ter-sync di Shuffle Anda. Heuristik `reversible_prefix_pairs` didokumentasikan di JSON, tetapi pada runtime action create yang tak terdaftar eksplisit dijatuhkan ke `needs_llm` (bukan menebak nama delete) agar tidak memetakan ke action yang tak ada.

9. Skema Knowledge Graph (Neo4j)

Node:

- `WORKFLOW {workflow_id, workflow_name, description, start_node}`
- `ACTION {action_id, workflow_id, label, app_name, action_name, app_id, app_version, role, position, is_start, parameters}`
- `APP {app_id, app_name, ...}`

- `(REVERSE_ACTION {rev_id, source_action_id, source_action_name, reverse_action_name, app_name, app_id, status, reason})`

Relasi:

- `(WORKFLOW)-[:CONTAINS]->(ACTION)`
- `(ACTION)-[:USES_APP]->(APP)`
- `(ACTION)-[:NEXT {condition}]->(ACTION)`
- `(ACTION)-[:HAS_REVERSE {status}]->(REVERSE_ACTION)`

Verifikasi cepat di Neo4j Browser (<http://localhost:7474>):

cypher

```
MATCH (a:ACTION)-[:HAS_REVERSE]->(r:REVERSE_ACTION)
RETURN a.label, r.reverse_action_name, r.status
```

Catatan: parameter disimpan sebagai property JSON pada `(ACTION)` (bukan node terpisah).

10. Validasi & Retry

`validators/validateWorkflow.js` menjalankan validasi berlapis lalu mengembalikan `review_required`:

- **Level A — struktural:** JSON valid, field wajib (name, description, actions), UUID v4, id unik, tepat satu `is_start_node`, `execution_delay` integer, `position`, kelengkapan branch.
- **Level B — semantik (Neo4j):** app_id/app_name/app_version cocok dengan katalog; action name valid untuk app-nya.
- **Level C — rule-based reverse mapping:** untuk action ber-status `auto_mapped`, reverse_action_name yang dipetakan **harus** muncul di hasil; jika tidak → error `REVERSE_MAPPING_MISMATCH`.
- **Human review flag:** action dengan `requires_manual_review: true` tidak dianggap error, tapi dikumpulkan ke `review_required`.

Jika ada error, `generateWithRetry` mengirim daftar error sebagai `retry_context` kembali ke llm-service agar LLM memperbaiki, hingga maksimal 3 percobaan. Kegagalan import Shuffle juga di-feedback dengan cara yang sama.

11. Hal yang WAJIB Diketahui (gotchas)

- 1. Jaringan eksternal `shared-platform` harus dibuat sebelum `up` (`docker network create shared-platform`)).
- 2. Ollama host harus konsisten: container `http://ollama:11434` atau host `http://host.docker.internal:11434`. Salah setel → generasi gagal/timeout.
- 3. Inference CPU lambat (5-15 menit/permintaan). Timeout sudah disetel besar; jangan kira hang. Pakai GPU bila bisa, lalu turunkan `OLLAMA_READ_TIMEOUT`.
- 4. Autentikasi internal pakai JWT: orchestrator login `admin`/`admin` ke `llm-service`. Ubah via `LLM_AUTH_USER/PASS` + `LLM_SECRET_KEY`.
- 5. Prompt dibangun di Python, bukan Node. Jangan memindahkannya jika ingin konsisten dengan paper.
- 6. Qdrant/RAG sudah dihapus total beserta fitur forward/recommend. Endpoint yang tersisa: `/api/reverse-workflow`, `/status/:id`, `/validate-workflow` (Node); `/api/v1/generate/reverse`, `/api/v1/generate/registry`, `/api/v1/auth/login`, `/health` (Python).
- 7. "reverse" = aksi kebalikan terpetakan + urutan dibalik, bukan sekadar menyalin action. Aksi tanpa pasangan/irreversibel → `requires_manual_review`, tidak dikarang.
- 8. Output selalu draft — sistem tidak mengeksekusi rollback. Analis wajib meninjau.
- 9. Sinkron app & import butuh Shuffle: tanpa `SHUFFLE_API_URL/KEY` yang valid, langkah sync di-skip (safe) dan import akan gagal — cocok untuk uji prompt/graf, tapi bukan end-to-end penuh.
- 10. `large_image` diisi server-side (Python) setelah generasi; LLM diminta mengisi `""`.
- 11. Menambah pasangan reverse = edit `config/reverseActionMap.json` saja.

12. Troubleshooting

Gejala	Kemungkinan penyebab	Solusi
<code>network shared-platform ... not found</code>	Jaringan eksternal belum dibuat	<code>docker network create shared-platform</code>
Generasi selalu timeout	Ollama host salah / model belum ditarik / CPU lambat	Set <code>OLLAMA_HOST</code> benar; <code>ollama pull llama3.1:8b</code> ; tunggu sesuai timeout
<code>[LLM] Login failed</code>	<code>LLM_AUTH_*</code> atau <code>LLM_SECRET_KEY</code> tidak	Samakan kredensial di <code>.env</code> , restart

Gejala	Kemungkinan penyebab	Solusi
	cocok	
No Action nodes found in Neo4j	Graph belum tersimpan / workflow_id salah	Pastikan Step 2-3 jalan; cek koneksi Neo4j
Import ke Shuffle gagal	SHUFFLE_API_URL/KEY salah atau Shuffle tak terjangkau	Perbaiki kredensial Shuffle; cek jaringan
REVERSE_MAPPING_MISMATCH berulang	LLM tidak mengeluarkan reverse_action_name terpetakan	Cek entri di reverseActionMap.json; lihat retry log
Neo4j auth error	NEO4J_PASSWORD di .env ≠ password container	Samakan password, atau reset container Neo4j

Lihat log: make logs atau docker compose logs -f reverse-workflow-service / llm-service.

13. Struktur Direktori (file kunci)

```
reverse-workflow-platform/
├─ docker-compose.yml           # 5 service, jaringan platform→shared-
platform
├─ .env.example                 # template environment
├─ Makefile                     # up/down/logs/pull-model/health/clean
├─ services/
│   └─ reverse-workflow-service/ # ORCHESTRATOR (Node, :5005)
│       ├── server.js           # endpoint utama + status + validate
│       ├── parsers/workflowParser.js
│       ├── graph/graphBuilder.js # bangun graph + REVERSE_ACTION/HAS_REVERSE
│       ├── neo4j/saveGraph.js   # simpan graph
│       ├── neo4j/queryWorkflowContext.js # baca graph + reverseMap
│       ├── llm/llmService.js    # login JWT, callLLM, generateWithRetry
│       ├── validators/validateWorkflow.js # validasi A/B/C + review flag
│       ├── builders/buildShuffleWorkflow.js # import ke Shuffle
│       ├── shuffleApps/...      # sinkron katalog app
│       └─ config/
│           ├── reverseActionMap.json # KAMUS reverse v2 per-app (edit di sini)
│           └─ reverseMap.js         # loader v2: resolveReverse(action, app) +
```

heuristik

```
└─ llm-service/                                # LLM SERVICE (Python/FastAPI, :8000)
  │ api/main.py                                # entrypoint, daftar router
  │ api/config.py                              # settings (tanpa qdrant/embedding)
  │ api/routes/generate.py                     # /generate/reverse + /generate/registry
  │ api/routes/other.py                        # /auth/login, /playbooks, /feedback,
/shuffle/status
  │ api/services/playbook_generator.py         # prompt reverse + Ollama
  │ api/services/graph_retrieval.py           # baca HAS_REVERSE dari Neo4j
  │ api/services/llm.py                        # klien Ollama
  └─ requirements.txt                          # tanpa qdrant-client/sentence-transformers
```

14. Keterbatasan

- Sistem menghasilkan **draft** reverse workflow; keputusan & eksekusi tetap di tangan analis.
- Kualitas untuk action ber-status `needs_llm` bergantung pada LLM; bila ragu, sistem menandai untuk peninjauan manual ketimbang mengarang.
- Cakupan pasangan reverse mengikuti `reverseActionMap.json` v2 (saat ini 16 app: FortiGate, AWS Network Firewall, Cisco ASA, Palo Alto, FortiEDR, Sophos Central, Microsoft Entra ID, Wazuh, Datadog, Elasticsearch, FortiSIEM, + app bawaan Shuffle). App di luar daftar / action tak terklasifikasi jatuh ke `needs_llm` atau `requires_manual_review`. Perluas dengan menambah entri per app.
- App yang tidak mengekspos aksi pembalik (mis. Palo Alto: tak ada unblock/delete rule) ditandai `requires_manual_review` — analis melengkapi saat review.
- End-to-end penuh (sync app + import) memerlukan instance Shuffle yang aktif.